

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

SC2006 LAB 1 Deliverables

Lab Group	**SCEF GRP 4**
Team	**URECAMatch**
Members	**1. IAN TAN** **2. ROBIN TEOH** **3. NICHOLAS NG** **4. EDWIN TAN QI LUN** **5. BENJAMIN NG**

1. Documentation of functional and non-functional requirements

A. Functional Requirements

FR1 : Administrator

1.1 The system shall allow an Admin to manage user accounts.

1.1.1 The system shall allow an Admin to create a new user account.

1.1.1.1 The system shall allow an Admin to assign a role (Student, Project Supervisor, Admin) to a user account.

1.1.1.2 The system shall restrict the creation of Admin-role accounts to existing Admins.

1.1.2 The system shall allow an Admin to view all user accounts.

1.1.2.1 The system shall allow an Admin to filter user accounts by role and account status.

1.1.3 The system shall allow an Admin to update a user account.

1.1.3.1 The system shall allow an Admin to update a user account's profile details.

1.1.3.2 The system shall allow an Admin to modify a user account's permissions.

1.1.4 The system shall allow an Admin to deactivate a user account.

1.1.4.1 When a user account is deactivated, the system shall terminate that user's active sessions.

1.2 The system shall allow an Admin to manage application cycles.

1.2.1 The system shall allow an Admin to create an application cycle.

1.2.1.1 The system shall allow an Admin to set the application opening date.

1.2.1.2 The system shall allow an Admin to set the application closing date.

1.2.1.3 The system shall allow an Admin to set the ranking-submission deadline for Project Supervisors.

1.2.1.4 The system shall reject an application cycle whose closing date is earlier than its opening date.

1.2.1.5 The system shall reject an application cycle whose dates overlap an existing application cycle.

1.2.2 The system shall allow an Admin to modify an application cycle.

1.2.2.1 The system shall only allow a date to be modified before that date has passed.

1.2.2.2 When the closing date of an active application cycle is modified, the system shall notify affected Students and Project Supervisors.

1.3 The system shall allow an Admin to execute the matching process.

1.3.1 The system shall allow an Admin to initiate the matching process.

1.3.1.1 The system shall only permit the matching process to be initiated after the application closing date and the ranking-submission deadline have passed.

1.3.1.2 The system shall prevent the matching process from being initiated more than once for the same application cycle.

1.3.2 The system shall produce a matching result in which no project exceeds its available slots.

1.3.3 The system shall produce a matching result in which no Student is assigned to more than one project.

1.3.4 The system shall exclude from the matching process any Student who has been registered to a self-proposed project.

1.4 The system shall allow an Admin to review and publish matching results.

1.4.1 The system shall allow an Admin to view the proposed matching results before they are published.

1.4.1.1 The system shall display the approval status of each proposed match.

1.4.2 The system shall allow an Admin to reassign a Student to a different project before results are published.

1.4.2.1 When an Admin reassigns a Student, the system shall update the affected projects' remaining slots.

1.4.2.2 When an Admin reassigns a Student, the system shall record the previous and new allocation.

1.4.3 The system shall allow an Admin to reassign a Student whose proposed match was rejected by a Project Supervisor.

1.4.4 The system shall allow an Admin to publish the matching results.

1.4.4.1 The system shall only allow an Admin to publish matches that have been approved by the relevant Project Supervisor.

1.4.4.2 When matching results are published, the system shall notify affected Students and Project Supervisors.

1.5 The system shall maintain an audit log of matching operations.

1.5.1 The system shall record each matching-process execution with its timestamp and initiating Admin.

1.5.2 The system shall record each manual reassignment with its timestamp and initiating Admin.

1.5.3 The system shall prevent any user, including an Admin, from modifying or deleting audit log entries.

1.5.4 The system shall allow an Admin to view the audit log.

FR2 : Student

2.1 The system shall allow a Student to search and browse project listings.

2.1.1 The system shall allow a Student to search for project listings by keyword.

2.1.2 The system shall allow a Student to filter project listings by school, topic tag, and Project Supervisor.

2.1.3 The system shall display a project listing's title, description, prerequisites, Project Supervisor, topic tags, and available slots.

2.1.4 If no project listing matches the search or filter criteria, the system shall indicate that no results were found.

2.2 The system shall allow a Student to apply for project listings.

2.2.1 The system shall allow a Student to submit an application to a project listing.

2.2.1.1 The system shall allow a Student to submit a statement of interest with an application.

2.2.1.2 The system shall prevent a Student from submitting more than one application to the same project listing.

2.2.1.3 The system shall prevent a Student from submitting an application after the application closing date.

2.2.1.4 When an application is submitted, the system shall notify the relevant Project Supervisor.

2.2.2 The system shall allow a Student to rank their submitted applications in order of preference.

2.2.2.1 The system shall allow a Student to modify their preference ranking before the application closing date.

2.2.3 The system shall allow a Student to withdraw an application.

2.2.3.1 The system shall only allow an application to be withdrawn before the application closing date.

2.3 The system shall allow a Student to complete and manage their profile.

2.3.1 The system shall require a Student to complete their profile after first sign-in.

2.3.2 The system shall allow a Student to enter their name, CGPA, year of study, and completed modules.

2.3.3 The system shall allow a Student to state their fields of research interest.

2.3.4 The system shall allow a Student to upload a supporting document.

2.3.4.1 The system shall accept supporting documents in PDF format up to 5 MB in size.

2.3.4.2 The system shall reject a supporting document that exceeds the permitted format or size.

2.3.5 The system shall allow a Student to view and edit their profile.

2.4 The system shall allow a Student to manage their matching outcome.

2.4.1 The system shall allow a Student to view the status of each of their applications.

2.4.1.1 If a Student has submitted no applications, the system shall indicate that no applications exist.

2.4.2 The system shall allow a Student to view their published match result.

2.4.3 The system shall allow a Student to accept a match offer.

2.4.3.1 When a Student accepts a match offer, the system shall withdraw that Student's other applications.

2.4.4 The system shall allow a Student to decline a match offer.

2.4.5 The system shall only allow a Student to accept or decline a match offer after the matching results have been published.

2.5 The system shall allow a Student to propose a self-proposed project.

2.5.1 The system shall allow a Student to create a self-proposed project proposal.

2.5.1.1 The system shall allow a Student to specify a proposal's title, description, and research interests.

2.5.1.2 The system shall allow a Student to select the Project Supervisor to whom the proposal is directed.

2.5.1.3 The system shall allow a Student to save a proposal as a draft.

2.5.2 The system shall allow a Student to submit a self-proposed project proposal to the selected Project Supervisor.

2.5.2.1 When a proposal is submitted, the system shall notify the selected Project Supervisor.

2.5.2.2 The system shall prevent a Student from submitting a proposal after the application closing date.

2.5.3 The system shall allow a Student to view the status of each submitted proposal.

2.5.4 The system shall prevent a Student who has been registered to a self-proposed project from submitting further applications or proposals.

FR3 : Project Supervisor

3.1 The system shall allow a Project Supervisor to manage research projects.

3.1.1 The system shall allow a Project Supervisor to create a new project listing.

3.1.1.1 The system shall allow a Project Supervisor to specify a project's title, description, and prerequisites.

3.1.1.2 The system shall allow a Project Supervisor to specify a project's number of available slots.

3.1.1.3 The system shall allow a Project Supervisor to add topic tags to a project.

3.1.2 The system shall allow a Project Supervisor to update a project listing.

3.1.2.1 The system shall only allow a project listing to be updated before the application closing date.

3.1.2.2 The system shall prevent a Project Supervisor from reducing a project's slots below the number of applicants already matched to it.

3.1.3 The system shall allow a Project Supervisor to close a project listing.

3.1.3.1 When a project listing is closed, the system shall remove it from Student search results.

3.2 The system shall allow a Project Supervisor to review and rank applicants.

3.2.1 The system shall allow a Project Supervisor to view the Students who applied to their project.

3.2.1.1 If no Student has applied to a project, the system shall indicate that there are no applicants.

3.2.2 The system shall allow a Project Supervisor to view an applicant's profile and supporting documents.

3.2.3 The system shall allow a Project Supervisor to rank applicants in order of preference.

3.2.3.1 The system shall allow a Project Supervisor to modify submitted rankings before the ranking-submission deadline.

3.2.4 The system shall allow a Project Supervisor to submit applicant rankings.

3.2.4.1 The system shall prevent the submission of applicant rankings after the ranking-submission deadline.

3.3 The system shall allow a Project Supervisor to confirm proposed matches.

3.3.1 The system shall allow a Project Supervisor to view the Students proposed to be matched to their project.

3.3.2 The system shall allow a Project Supervisor to approve a proposed match.

3.3.2.1 When a proposed match is approved, the system shall mark it as eligible for publication by an Admin.

3.3.3 The system shall allow a Project Supervisor to reject a proposed match.

3.3.3.1 When a proposed match is rejected, the system shall record the rejection for Admin review.

3.3.3.2 When a proposed match is rejected, the system shall return the affected project slot to available.

3.3.4 The system shall only allow a Project Supervisor to approve or reject a proposed match after the matching process has been executed.

3.3.5 The system shall prevent a Project Supervisor from approving or rejecting a proposed match after the matching results have been published.

3.3.6 The system shall prevent a Project Supervisor from assigning a Student to a project listing outside the matching process.

3.4 The system shall allow a Project Supervisor to manage self-proposed projects.

3.4.1 The system shall allow a Project Supervisor to view self-proposed project submissions directed to them.

3.4.1.1 If no self-proposed project has been submitted to a Project Supervisor, the system shall indicate that there are no submissions.

3.4.2 The system shall allow a Project Supervisor to view a proposing Student's profile, research interests, and supporting documents.

3.4.3 The system shall allow a Project Supervisor to accept a self-proposed project.

3.4.3.1 When a self-proposed project is accepted, the system shall register the Student to that project.

3.4.3.2 The system shall prevent a Project Supervisor from accepting a self-proposed project after the matching process has been executed.

3.4.4 The system shall allow a Project Supervisor to decline a self-proposed project.

3.4.4.1 When a self-proposed project is declined, the system shall record the outcome and notify the proposing Student.

FR4 : Users

4.1 The system shall allow a user to register for an account.

4.1.1 The system shall allow a user to register with an email address and password.

4.1.1.1 The system shall restrict registration to email addresses ending in the NTU domain (@e.ntu.edu.sg or @ntu.edu.sg), or approved external domains.

4.1.1.2 The system shall reject a registration using an email address already associated with an existing account.

4.1.2 The system shall send a verification code to the registered email address.

4.1.2.1 The system shall require a user to enter the correct verification code to activate the account.

4.1.2.2 The system shall reject a verification code that is incorrect.

4.1.2.3 The system shall reject a verification code that has expired.

4.1.2.4 The system shall allow a user to request a new verification code.

4.2 The system shall allow a verified user to sign in.

4.2.1 The system shall allow a verified user to sign in with their email address and password.

4.2.1.1 The system shall mask the password input, with an option to reveal it.

4.2.1.2 If the email and password do not match, the system shall display an error message.

4.2.1.3 The system shall prevent sign-in to a deactivated account.

4.2.2 The system shall allow a signed-in user to sign out.

4.2.3 The system shall end a user's session after 30 minutes of inactivity.

4.3 The system shall allow a user to reset a forgotten password.

4.3.1 The system shall send a password-reset link to a registered email address on request.

4.3.2 The system shall allow a user to set a new password using a valid reset link.

4.3.3 The system shall reject a password-reset link that has expired or has already been used.

4.4 The system shall notify users of relevant events.

4.4.1 The system shall create a notification record when a triggering event occurs.

4.4.1.1 The system shall record each notification with its recipient, message, and timestamp.

4.4.1.2 The system shall deliver a notification only to the user it concerns.

4.4.2 The system shall notify a Student when the status of their application changes.

4.4.3 The system shall notify a Student when their match result is published.

4.4.4 The system shall notify a Project Supervisor when a new application is submitted to their project listing.

4.4.5 The system shall notify affected Students and Project Supervisors when the closing date of an active application cycle is modified.

4.4.6 The system shall allow a user to view their notifications.

4.4.6.1 The system shall display a user's notifications in reverse chronological order.

4.4.6.2 The system shall indicate which notifications are unread.

4.4.6.3 The system shall mark a notification as read when the user views it.

4.4.6.4 If a user has no notifications, the system shall indicate that none exist.

B. Non-Functional Requirements

Category	Description
Security	**Authentication** ● Registration shall be restricted to NTU email addresses (@e.ntu.edu.sg or @ntu.edu.sg), or approved external domains. ● A User session shall expire after 30 minutes of inactivity. **Authorization** ● Every request shall be authorized against the requesting User's role, and any attempt to access a function outside that role shall be denied. ● A Student shall only be able to view their own profile, applications, and match result, without access to the profiles or results of any other Student. ● A Project Supervisor shall only be able to view applicant profiles and Supporting Documents for Students who applied to the URECA Project Listings posted by them.

Reliability	**System Availability** ● The portal shall maintain 99.9% uptime during the official application and matching periods. **Data Persistence** ● All user preference and ranking data must be committed to persistent storage upon submission to ensure zero data loss in the event of a system crash.
Performance	**Responsiveness** ● During the active registration period, all URECA projects (or a subset sufficient for loading a preliminary table display) shall be queried and fetched within 5 seconds. ● Basic visual feedback of interactive elements on the user interface shall be displayed within no more than 2 seconds after user interaction. **Notification Latency** ● In-app notifications shall appear within a minute of the relevant update, while email notifications (if implemented) shall be dispatched within a maximum of 15 minutes to the receiving users.
Maintainability	**Service Recovery** ● Following a system restart or unexpected crash, all core portal services and API integrations must recover and become accessible within 5 minutes. **Maintainability, Reusability & Configurability** ● The system shall allow for reusability across multiple application cycles taking place at the start of each academic year with simple configuration changes and minimal/no code modifications. **Documentation** ● The system shall be accompanied by maintained code documentation, API specification, and setup guide sufficient for a new developer to build and run it. **Modularity & Extensibility** ● The system shall be implemented with good software engineering practices in mind, featuring a layered architecture that separates model, controller, and view.

	● The system shall be implemented with future extensibility in mind e.g. the addition of new account types, topic tags, project description attributes shall all require minimal code changes.
Scalability	**<u>Scalability</u>** ● The system shall accommodate up to 1000 concurrent users without significant service degradation.
Usability	**<u>User Interface</u>** ● The user interface must be fully responsive, rendering correctly on common screen sizes (desktop, tablet, mobile) without content clipping. **<u>Visual Feedback:</u>** ● The system must provide visual feedback (e.g., loading spinners, progress bars, or "Success/Error" notifications) for all interactive elements, such as form submissions or ranking updates. **<u>Error Clarity</u>** ● All error messages displayed to the user must be written in plain, non technical language and provide clear instructions on how to resolve the issue (e.g., "Invalid file format: Please upload a PDF file less than 5MB").

2. Data Dictionary

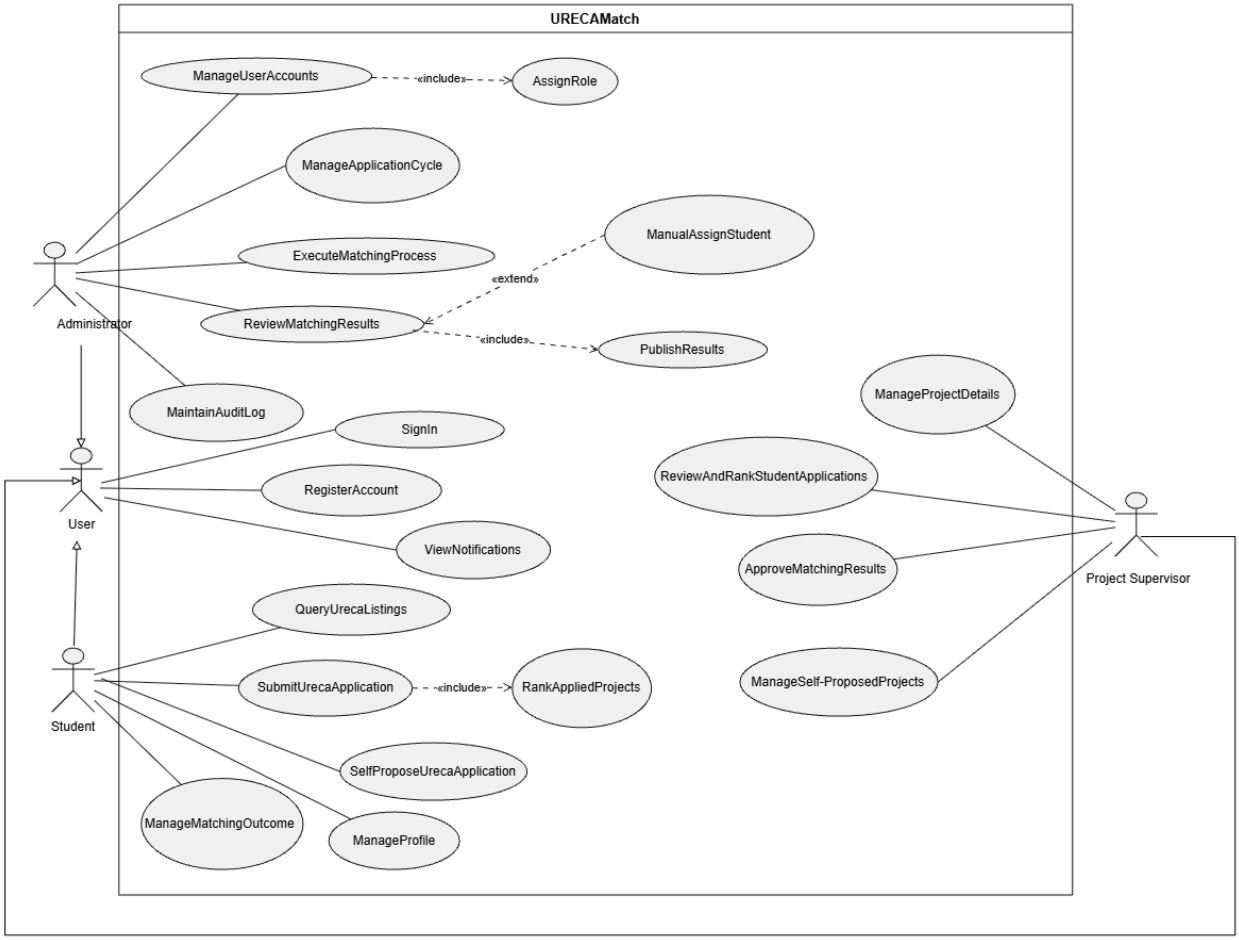
Term	**Definition**
URECA	Undergraduate Research Experience on Campus, a programme in which NTU undergraduate students undertake a research project supervised by a faculty staff or external mentor. Such projects can either be listed by the mentors, or self-proposed from the student.
URECAMatch (aka. the “System”)	The system specified in this technical requirements document. Its primary goal is to provide a platform to facilitate an improved application process for Students, Project Supervisors, and Admins.

URECA Project Listing	A URECAMatch descriptive entry of a URECA Project put forward by a Project Supervisor, and made available on the platform for Students to apply for. Comprises a project title, project description, (optional) prerequisites, topic tags, offering faculty/organization and main research location.
URECA Project	A research project carried out by a Student under the mentorship of a Project Supervisor, typically over the span of 11 months in an NTU academic year. Typically, URECA Projects are put forwards by a Project Supervisor, but Students can also initiate a Self-Proposed URECA Project
Self-Proposed URECA Project (aka. “Self-Initiated URECA Project”)	A URECA Project proposed by a Student instead of being put forward by a Project Supervisor. Students can propose a URECA Project by submitting an Application for a Self-Proposed URECA Project, and subsequently contacting a Project Supervisor to approve the supervision of the project and/or provide additional resources if necessary.
User/Actor	A person operating an account on the URECAMatch platform. A User can be classified into 3 different types: Admin, Project Supervisor, and Student.
Admin (aka. “Administrator”)	One of the 3 primary Users/Actors of the URECAMatch platform. Typically a member of the administrative staff overseeing the URECA Programme, the Admin shall have system-wide administrative privileges to manage Student and Project Supervisor accounts, configure application opening and closing dates, manually match projects to Students/Supervisors, notify Users, and modify various other records within the URECAMatch system.
Project Supervisor	One of the 3 primary Users/Actors of the URECAMatch platform. A Project Supervisor acts as the mentor in a URECA Project, and is able to supervise multiple URECA Projects and put forward these projects as URECA Project Listings on the platform. Project Supervisors are chiefly NTU faculty teaching staff, but can hail from external organizations as well.
Student	One of the 3 primary Users/Actors of the URECAMatch platform. A Student is a full-time NTU undergraduate who meets all eligibility requirements to be invited for the

	URECA application window, and can be accepted to at most one concurrent URECA Project.
Application	A Student's request to be considered for a URECA Project.
NTU	Nanyang Technological University, a tertiary higher education institution in Singapore. It hosts the URECA programme for its eligible undergraduates.
GPA	Grade Point Average, the aggregate measure of a student's academic performance across all enrolled semesters on a scale from 0.00 to 5.00, with 5.00 being the highest numerical grade point possible.
Prerequisite	A condition stated on a URECA Project Listing describing the necessary background that a Student shall possess in order to be considered for a given URECA Project.
Topic Tags	A short, key-word label attached to a listing (eg. "Artificial Intelligence" or "Finance") that highlights the topics involved in the URECA Project. May be used for filtering search results.
Statement of Interest	A brief free-text personal statement written by a Student when submitting an Application for a URECA Project Listing. Usually used to explain their motivation in applying for the project, as well as other relevant details not captured by the Supporting Documents.
Supporting Document	A file uploaded by a Student in support of their Application, e.g. academic transcripts, resume, or CV.
Application Cycle	A defined period set by Admins, in which Students can apply for projects listed by Project Supervisors, and Project Supervisors can consider and accept Students into their projects. During this period, Project Supervisors can also list new URECA Project Listings, and Students can similarly submit proposals for Self-Proposed URECA Projects.

3. Initial Use Case Model (Use Case diagram and Use Case descriptions)

A. Use Case Diagram



B. Use Case Descriptions

I. For Functional Requirement #1 (Admin)

I.I ManageApplicationCycle

Use Case ID:	#1-1
Use Case Name:	ManageApplicationCycle

Created By:	Nicholas Ng	**Last Updated By:**	Nicholas Ng
Date Created:	26th August 2026	**Date Last Updated:**	26th August 2026

Actor:	Admin
Description:	Admin creates a new Application Cycle or modifies an existing one, configuring time parameters (start date, end date) and other settings for URECAMatch.
Preconditions:	The Admin is signed in.
Postconditions:	The Application Cycle is successfully created or updated in URECAMatch, and operates in accordance with the configured parameters.
Priority:	High
Frequency of Use:	Several times per year.
Flow of Events:	1. The Admin navigates to the Application Cycles tab and selects either "New Application Cycle" or an existing cycle to modify. 2. The System displays the Application Cycle configuration page. 3. The Admin configures or modifies the cycle parameters, including the name, start date, and end date. 4. The Admin clicks “Create” or “Save” to submit the configuration. 5. The System validates the provided parameters and displays a confirmation prompt. 6. The Admin clicks “Confirm” to proceed. 7. The System saves the Application Cycle and updates the active records.
Alternative Flows:	AF-1: The Admin clicks “Cancel” 1. The System confirms the cancellation via a prompt. 2. The Admin acknowledges the cancellation. 3. The System discards filled fields or changes and exits to the main page.

Exceptions:	E1: The closing date of an Application Cycle is earlier than the start date. 1. The System rejects this change. 2. The System displays the rejection reason to the Admin, and remains on the configuration page. E2. The Admin attempts to create a new Application Cycle with dates that overlap an existing Application Cycle. 1. The System rejects this change. 2. The System displays the rejection reason to the Admin, and remains on the configuration page. E3. The Admin attempts to create a new Application Cycle with a closing date that has already lapsed 1. The System rejects this change. 2. The System displays the rejection reason to the Admin, and remains on the configuration page. E4: The Admin made no changes to the Application Cycle. 1. The System greys out the “Save” button and disables its interactivity.
Includes:	-
Special Requirements:	If the closing date of an Application Cycle is modified during its active period, it shall prompt the Admin to send out a notification to affected Users.
Assumptions:	Only one Application Cycle shall be active at any given time.
Notes and Issues:	-

I.II ExecuteMatchingProcess

Use Case ID:	#1-2		
Use Case Name:	ExecuteMatchingProcess		
Created By:	Edwin Tan	**Last Updated By:**	Edwin Tan
Date Created:	29th August 2026	**Date Last Updated:**	29th August 2026

Actor:	Admin

Description:	Admin runs the stable-matching algorithm to assign students to projects based on ranked preferences and supervisor rankings
Preconditions:	1. The Admin is signed in. 2. The application closing date and supervisor ranking deadline have lapsed.
Postconditions:	A proposed allocation is generated where no project exceeds available capacity and each student receives at most one match.
Priority:	High
Frequency of Use:	Once per year.
Flow of Events:	1. The Admin navigates to the "Matching Round" dashboard. 2. The System displays cycle status and matching controls. 3. The Admin selects "Execute Matching". 4. The System runs the matching algorithm against rankings and verifies capacity constraints. 5. The System commits match results to storage and logs an immutable entry with Admin ID and timestamp. 6. The System displays summary metrics including matched students, unmatched count, and average preference rank.
Alternative Flows:	-
Exceptions:	E1: Matching initiated prior to deadline closure 1. The System rejects execution and informs the Admin that applications or supervisor ranking periods remain active. E2: Process already executed in the current cycle

	1. The System blocks redundant execution to prevent duplicate match generation.
Includes:	-
Special Requirements:	-
Assumptions:	All submitted rankings are final upon deadline lapse.
Notes and Issues:	-

I.III ReviewMatchingResults

Use Case ID:	#1-3		
Use Case Name:	ReviewMatchingResults		
Created By:	Edwin Tan	**Last Updated By:**	Edwin Tan
Date Created:	29th August 2026	**Date Last Updated:**	29th August 2026

Actor:	Admin
Description:	Admin verifies supervisor approvals, handles manual reassignments, and publishes finalized matching outcomes to students and supervisors.
Preconditions:	1. The Admin is signed in. 2. The automated matching process has been executed. 3. Proposed matches exist in the system.
Postconditions:	Approved matches are published, slots are updated, and users can view their final outcomes.
Priority:	High
Frequency of Use:	Medium (Up to Multiple Times per Student)

Flow of Events:	1. The Admin navigates to the "Matching Round" page. 2. The System displays the proposed matches along with supervisor approval statuses. 3. The Admin confirms that matches have been approved by the respective Project Supervisors. 4. The Admin clicks "Publish Results to Users". 5. The System presents a confirmation dialog. 6. The Admin clicks "Confirm". 7. The System transitions matching results to the published state and triggers automated notifications to students.
Alternative Flows:	AF-S3: Manual Reassignment of Student 1. The Admin selects "Reassign" next to a proposed match entry. 2. The System displays available alternative projects with remaining slots. 3. The Admin selects an alternate project listing and confirms. 4. The System decrements available slots on the new project and increments slots on the original project. 5. The System logs the previous allocation, new allocation, initiating Admin, and timestamp to the audit log. 6. The System refreshes the proposed match list.
Exceptions:	E1: Admin attempts to publish unapproved matches 1. The System prevents publishing and displays pending supervisor approvals requiring resolution.
Includes:	PublishResults
Extends:	ManualAssignStudent
Special Requirements:	Reassignment operations must be atomic to prevent slot over-allocation.

Assumptions:	Project Supervisors complete their match approval reviews before publishing.
Notes and Issues:	Manual overrides must be logged before final release.

I.IV ManageUserAccounts

Use Case ID:	#1-4		
Use Case Name:	ManageUserAccounts		
Created By:	Edwin Tan	**Last Updated By:**	Edwin Tan
Date Created:	29th August 2026	**Date Last Updated:**	29th August 2026

Actor:	Admin
Description:	Admin creates, modifies, updates permissions for, or deactivates user accounts in the system
Preconditions:	The Admin is signed in.
Postconditions:	The user's account details, assigned roles, or activation status are updated in persistent storage.
Priority:	High
Frequency of Use:	Several times per week

Flow of Events:	1. The Admin navigates to the "Manage Users" dashboard. 2. The System displays the full directory of users with role and status filters. 3. The Admin selects a specific user account to edit. 4. The Admin updates the user's role (Student, Project Supervisor, Admin) or modifies account permissions. 5. The Admin clicks "Save". 6. The System commits the changes and logs the action.
Alternative Flows:	AF-S3: Deactivate User Account: 1. The Admin selects "Suspend" or "Remove" next to an active user. 2. The System prompts for confirmation. 3. The Admin confirms, and the System marks the user account as inactive.
Exceptions:	E1: Creation of Admin role by unauthorized user: 1. The System rejects the role change if the current session does not hold Admin privileges.
Includes:	AssignRole
Special Requirements:	Admin account creation and role elevation operations must be strictly restricted to existing authenticated Admins.
Assumptions:	All registered user accounts have valid NTU domain emails.
Notes and Issues:	Deactivated users shall have their active sessions immediately terminated.

I.V MaintainAuditLog

Use Case ID:	#1-5

Use Case Name:	MaintainAuditLog		
Created By:	Edwin Tan	**Last Updated By:**	Edwin Tan
Date Created:	29th August 2026	**Date Last Updated:**	29th August 2026

Actor:	Admin
Description:	Admin views immutable system audit records of matching execution runs and manual allocation overrides.
Preconditions:	The Admin is signed in.
Postconditions:	Audit log entries are retrieved and rendered in read-only mode
Priority:	Medium
Frequency of Use:	After matching cycles or as needed for review
Flow of Events:	1. The Admin navigates to the "Audit & Overrides" tab. 2. The System displays timestamped records of all matching runs and manual student reassignments along with initiating Admin IDs. 3. The Admin filters or inspects specific log entries.
Alternative Flows:	-
Exceptions:	E1: Modification attempt: 1. The System strictly prevents any edit or delete operations on audit records.
Includes:	-
Special Requirements:	Audit log entries must be write-once, append-only, and completely immutable to all users including Admins.
Assumptions:	Every automated matching execution and manual reassignment action automatically triggers an audit log write.

Notes and Issues:	Log entries must persist across cycle resets for compliance and tracking.

I.VI ManualAssignStudent

Use Case ID:	#1-6		
Use Case Name:	ManualAssignStudent		
Created By:	Nicholas Ng	**Last Updated By:**	Nicholas Ng
Date Created:	29th August 2026	**Date Last Updated:**	29th August 2026

Actor:	Admin
Description:	Admin manually assigns a Student to a project. This function shall be reserved for when exceptional circumstances mandate a manual override of URECA Project assignments.
Preconditions:	The Admin is signed in.
Postconditions:	Students are manually assigned to a URECA Project.
Priority:	High
Frequency of Use:	Low (A few times per Application Cycle)
Flow of Events:	1. Admin reviews the matching round results. 2. Admin clicks on “Manual Assign” next to a Student’s entry. 3. Admin selects an available URECA Project to be assigned to Student. 4. Admin clicks “Confirm”.
Alternative Flows:	-
Exceptions:	E1: Student is already assigned to a URECA Project

	1. The System displays a warning message, and the URECA Project is not assigned. 2. Admin unassigns the current URECA Project from Student. 3. Admin repeats the ManualAssignStudent use case from Step 2.
Includes:	-
Extends:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

II. For Functional Requirement #2 (Student)

II.I QueryUrecaListings

Use Case ID:	#2-1		
Use Case Name:	QueryUrecaListings		
Created By:	Nicholas Ng	**Last Updated By:**	Nicholas Ng
Date Created:	26th August 2026	**Date Last Updated:**	26th August 2026

Actor:	Student
Description:	Student browses and queries available URECA Project Listings using keyword search and/or filters.
Preconditions:	1. The Student is signed in. 2. An Application Cycle is active.
Postconditions:	-
Priority:	High
Frequency of Use:	Very High

Flow of Events:	1. The Student selects “View Available Projects” 2. The System displays the first page of available URECA Project Listings in a table view, with certain key information like title, offering organization, and Project Supervisor displayed. 3. The Student enters a keyword. 4. The Student clicks the “Search” button. 5. The System returns listings that match the query.
Alternative Flows:	AF-S3: The Student uses additional filters/search criteria to query the listings. 1. The Student chooses to filter by topic tags, faculty/offering organization, or prerequisites. 2. The System returns listings that match all search/filter criteria. AF-S6: The Student chooses to return to an unfiltered view of the listings. 1. The Student clears all search criteria and clicks “Search”. 2. The System returns to the default and unfiltered paginated view of all URECA Projects Listings.
Exceptions:	E1: The search request from the server times out. 1. The System informs the Student accordingly via an error message.
Includes:	-
Special Requirements:	Each query request shall complete within 5 seconds.
Assumptions:	There will be ample URECA Project Listings posted by Project Supervisors during an active Application Cycle.
Notes and Issues:	-

II.II SubmitUrecaApplication

Use Case ID:	#2-2		
Use Case Name:	SubmitUrecaApplication		
Created By:	Nicholas Ng	**Last Updated By:**	Nicholas Ng

Date Created:	26th August 2026	**Date Last Updated:**	26th August 2026

Actor:	Student
Description:	Student applies for a URECA Project by submitting an Application to the desired URECA Project Listing
Preconditions:	1. Students are signed in. 2. Application Cycle is Active. 3. At least one URECA Project Listing is available for application. 4. Student has completed their profile (including transcript upload).
Postconditions:	An Application for the desired URECA Project has been submitted to a Project Supervisor for consideration.
Priority:	High
Frequency of Use:	Medium (a few times per Student)
Flow of Events:	1. Students view the full details of a URECA Project Listing. 2. Students click "Apply". 3. Students write a Statement of Interest and upload supporting documents. 4. Students rank the project in order of preference (RankAppliedProjects). 5. Students click "Submit". 6. The System validates the submitted inputs and documents. 7. The System submits the application for Project Supervisor consideration.
Alternative Flows:	AF-S3: Student clicks “Back to Project” 1. The System displays a confirmation prompt about unsaved Application data. 2. Student clicks “Confirm”. 3. The System returns to displaying the full descriptive view of the selected URECA Project. AF-S5: Student clicks “Save as Draft”

	1. The System saves current text in Statement of Interest and uploaded documents as a draft. 2. After saving, the System displays feedback to the Student, and remains on the Application page.
Exceptions:	E1: Student leaves Statement of Interest blank. 1. The System displays the error message to the Student upon clicking “Submit”, and remains on the Application page. E2: Student uploads documents that are not in PDF form. 1. The System rejects the upload while displaying the appropriate error feedback to the Student. E3: Application Cycle ends while Students are editing their application before submission. 1. The Student clicks “Submit” 2. The System rejects the submission and displays the reason for rejection. 3. The System returns to displaying the full details of the selected URECA Project, while greying and disabling interactivity of the “Apply” button.
Includes:	RankAppliedProjects
Special Requirements:	Statement of Interest text field shall be programmatically restricted to the max length allowable for that Application Cycle (eg. 300 words). This can be modified by the Admin.
Assumptions:	-
Notes and Issues:	-

II.III SelfProposeUrecaApplication

Use Case ID:	#2-3		
Use Case Name:	SelfProposeUrecaApplication		
Created By:	Nicholas Ng	**Last Updated By:**	Nicholas Ng
Date Created:	26th August 2026	**Date Last Updated:**	26th August 2026

Actor:	Student

Description:	Students self-propose a URECA Project and submit it to their desired Project Supervisor for approval.
Preconditions:	1. Students are signed in. 2. Application Cycle is currently active.
Postconditions:	A self-proposed URECA Application has been submitted for Project Supervisor consideration.
Priority:	High
Frequency of Use:	Low (Most Students will apply to listed URECA Projects)
Flow of Events:	1. Student clicks on “Propose a Project”. 2. The System displays the Application page for a Self-Proposed URECA Project. 3. Students fill up their proposed URECA Project title, writeup, research interests, supporting documents, and choice of Project Supervisor. 4. Students click “Submit”. 5. The System validates all required fields. 6. The System submits the Application for Project Supervisor consideration.
Alternative Flows:	AF-S3: Student clicks on the “Browse Projects” tab without submitting the Application. 1. The System displays a confirmation prompt about unsaved Application data. 2. Student clicks “Confirm”. 3. The System returns to displaying the URECA Project Listings in table view. AF-S5: Student clicks “Save as Draft” 1. The System saves current text in Statement of Interest and uploaded documents as a draft. 2. After saving, the System displays feedback to the Student, and remains on the Application page.
Exceptions:	E1: Student leaves one or more required field(s) blank. 1. The System displays the error message to the Student upon clicking “Submit”, and remains on the Application page. E2: Student uploads documents that are not in PDF form. 2. The System rejects the upload while displaying the appropriate error feedback to the Student.

	E3: Application Cycle ends while Students are editing their application before submission. 1. The Student clicks “Submit” 2. The System rejects the submission and displays the reason for rejection. 3. The System returns to displaying the full details of the selected URECA Project, while greying and disabling interactivity of the “Submit” button.
Includes:	-
Special Requirements:	All free-text fields shall be programmatically restricted to the max length allowable for that Application Cycle (eg. 300 words for Statement of Interest or 500 words for URECA Project description). This can be modified by the Admin for each Application Cycle.
Assumptions:	-
Notes and Issues:	-

II.IV ManageMatchingOutcome

Use Case ID:	#2-4		
Use Case Name:	ManageMatchingOutcome		
Created By:	Edwin Tan	**Last Updated By:**	Edwin Tan
Date Created:	29th August 2026	**Date Last Updated:**	29th August 2026

Actor:	Student
Description:	Students review the published matching allocation and decide whether to accept or decline the offered project slot.
Preconditions:	1. The Student is signed in. 2. Matching results have been published by an Admin.

Postconditions:	Match status is recorded as accepted or declined; accepting automatically closes out all other active applications
Priority:	High
Frequency of Use:	Once per year.
Flow of Events:	1. The Student navigates to "My Applications". 2. The System displays the "Matching Round Complete" banner and project match offer. 3. The Student reviews the project title and assigned Project Supervisor. 4. The Student clicks "Accept Match". 5. The System displays a warning modal stating that accepting will withdraw all other pending applications. 6. The Student selects "Confirm". 7. The System records the acceptance and automatically sets all other applications submitted by the Student to withdraw.
Alternative Flows:	AF-S4: Student declines match offer 1. The Student clicks "Decline". 2. The System displays a confirmation dialog warning that the match slot will be forfeited. 3. The Student clicks "Confirm". 4. The System marks the allocation status as declined.
Exceptions:	E1: Student attempts to access outcomes before Admin publication 1. The System shows current application submission status and hides the offer acceptance interface.
Includes:	ViewMatchingResult
Special Requirements:	Immediate in-app visual confirmation upon selection.

Assumptions:	Each student receives at most one match offer per cycle.
Notes and Issues:	Once confirmed, acceptance cannot be undone through the student UI.

II.V ManageProfile

Use Case ID:	#2-5		
Use Case Name:	ManageProfile		
Created By:	Edwin Tan	**Last Updated By:**	Edwin Tan
Date Created:	29th August 2026	**Date Last Updated:**	29th August 2026

Actor:	Student
Description:	Student completes, views, and updates personal academic details, completed modules, research interests, and transcript documents.
Preconditions:	The Student is signed in.
Postconditions:	Updated profile data and uploaded transcript are saved to the student record.
Priority:	High
Frequency of Use:	Upon first sign-in and occasionally updated.

Flow of Events:	1. The Student navigates to "My Profile". 2. The System displays the student's name, GPA, year of study, completed modules, and fields of research interest. 3. The Student updates fields and uploads an updated academic transcript PDF. 4. The Student clicks "Save changes". 5. The System validates the file and inputs, then saves the profile.
Alternative Flows:	-
Exceptions:	E1: Invalid transcript format or file size > 5 MB: 1. The System rejects the upload and displays an error message.
Includes:	UploadTranscript
Special Requirements:	File upload must be restricted to PDF format with a maximum size limit of 5 MB
Assumptions:	First-time sign-in requires completion of the profile before browsing or applying for projects
Notes and Issues:	GPA and module data may optionally be extracted from the transcript in future releases

III. For Functional Requirement #3 (Project Supervisor)

III.I ManageProjectDetails

Use Case ID:	#3-1		
Use Case Name:	ManageProjectDetails		
Created By:	Robin Teoh	**Last Updated By:**	Robin Teoh

Date Created:	26th August 2026	**Date Last Updated:**	26th August 2026

Actor:	Project Supervisor
Description:	Allows a Project Supervisor to create, update, and close research project listings.
Preconditions:	Project Supervisor is logged in and authenticated.
Postconditions:	The project listing is created, updated, or closed, and its current state is saved in the system.
Priority:	High
Frequency of Use:	Medium
Flow of Events:	1. The system displays the Project Supervisor's existing project listings and prompts them to select CREATE, UPDATE, or CLOSE. 2. If the Project Supervisor selects CREATE, the system prompts for the project title, description, and prerequisites. 3. The Project Supervisor enters the number of available slots. 4. The Project Supervisor adds topic tags to the project. 5. The Project Supervisor selects CONFIRM. 6. The system validates the entered data, saves the project listing, and displays it as published.
Alternative Flows:	AF-S2: If the Project Supervisor selects UPDATE 1. The system displays the selected project's existing details for editing. 2. The Project Supervisor modifies the details and selects SAVE. 3. The system validates and saves the updated listing. 4. If the application closing date has passed, the system rejects the update and displays a message that listings can no longer be modified. AF-S2: If the Project Supervisor selects CLOSE

	1. The system prompts the Project Supervisor to confirm closing the project. 2. On confirmation, the system marks the project listing as closed and removes it from student browsing. AF-S6: If the Project Supervisor selects CANCEL 1. The system discards the entered data and returns to the project listing screen.
Exceptions:	E1**:** A required field is missing or invalid. 1. The system rejects the submission. 2. The system displays an error message identifying the invalid field and prompts the Project Supervisor to correct it. 3. The system remains on the project configuration page. E2: The Project Supervisor attempts to reduce the number of slots below the number of applicants already matched to the project. 1. The system rejects the change. 2. The system displays an error message stating the minimum permitted number of slots. 3. The system remains on the project configuration page.
Includes:	-
Special Requirements:	The system must validate all user input data.
Assumptions:	The current matching cycle and its deadlines have been configured by an Administrator.
Notes and Issues:	None

III.II ReviewAndRankStudentApplications

Use Case ID:	#3-2		
Use Case Name:	ReviewAndRankStudentApplications		
Created By:	Robin Teoh	**Last Updated By:**	Robin Teoh
Date Created:	26th August 2026	**Date Last Updated:**	26th August 2026

Actor:	Project Supervisor
Description:	Allows a Project Supervisor to review students who applied to their project and rank them in order of preference.
Preconditions:	1. Project Supervisor is logged in and authenticated. 2. The Project Supervisor has at least one published project listing. 3. At least one student has applied to the project.
Postconditions:	The Project Supervisor's applicant rankings are submitted and recorded for use in the matching process.
Priority:	High
Frequency of Use:	Medium
Flow of Events:	1. The Project Supervisor selects a project to review. 2. The system displays the list of students who have applied to that project. 3. The Project Supervisor selects an applicant to review. 4. The system displays the applicant's profile and transcript. 5. The Project Supervisor returns to the applicant list and arranges the applicants in order of preference. 6. The Project Supervisor selects SUBMIT RANKINGS. 7. The system saves the rankings and confirms submission.

Alternative Flows:	AF-S6: If the Project Supervisor modifies rankings after an earlier submission 1. The system replaces the previously submitted rankings with the new order, provided the ranking-submission deadline has not passed. AF-S7: If the Project Supervisor selects CANCEL 1. The system discards the unsaved ranking order and returns to the applicant list.
Exceptions:	E1: The Project Supervisor attempts to submit rankings after the ranking-submission deadline has passed. 1. The system rejects the submission. 2. The system displays a message stating that the ranking-submission deadline has closed. 3. The system remains on the applicant list page. E2: No Student has applied to the selected project. 1. The system displays a message stating that there are no applicants to rank. 2. The system disables the ranking and submission functions for that project.
Includes:	-
Special Requirements:	Applicant transcripts must be viewable within the system without requiring external software.
Assumptions:	Students have completed their profiles, including uploading a transcript, before applying.
Notes and Issues:	Rankings must be submitted before the deadline set by the Administrator; supervisors who do not submit rankings may have their applicants matched on student preference alone.

III.III ApproveMatchingResults

Use Case ID:	#3-3		
Use Case Name:	ApproveMatchingResults		
Created By:	Robin Teoh	**Last Updated By:**	Robin Teoh
Date Created:	26th August 2026	**Date Last Updated:**	26th August 2026

Actor:	Project Supervisor
Description:	Allows a Project Supervisor to review students proposed to be matched to their project and approve or reject each proposed match.
Preconditions:	1. Project Supervisor is logged in and authenticated. 2. The Administrator has executed the matching process. 3. At least one proposed match exists for the Project Supervisor's project.
Postconditions:	Each proposed match is recorded as approved or rejected. Approved matches become eligible for publication by the Administrator.
Priority:	High
Frequency of Use:	Low
Flow of Events:	1. The system displays the students proposed to be matched to the Project Supervisor's project. 2. The Project Supervisor selects a proposed match to review. 3. The system displays the student's profile and application details. 4. The Project Supervisor selects APPROVE. 5. The system records the match as approved and marks it as eligible for publication. 6. Steps 3 to 6 are repeated until all proposed matches have been processed.

Alternative Flows:	AF-S5: If the Project Supervisor selects REJECT 1. The system prompts the Project Supervisor to confirm the rejection. 2. On confirmation, the system records the match as rejected. 3. The system records the rejection for Administrator review. 4. Return to step 3. AF-S5: If the Project Supervisor selects CANCEL 1. The system returns to the list of proposed matches without recording a decision.
Exceptions:	E1: The Project Supervisor attempts to approve or reject a proposed match after the matching results have been published. 1. The system prevents the approval or rejection. 2. The system displays a message stating that the matching results are final. 3. The system displays the proposed matches in a read-only state.
Includes:	-
Special Requirements:	None
Assumptions:	The Administrator will review recorded rejections and reassign affected students before publishing results.
Notes and Issues:	None

III.IV ManageSelf-ProposedProjects

Use Case ID:	#3-4
Use Case Name:	ManageSelf-ProposedProjects

Created By:	Edwin Tan	**Last Updated By:**	Edwin Tan
Date Created:	29th August 2026	**Date Last Updated:**	29th August 2026

Actor:	Project Supervisor
Description:	Allows a Project Supervisor to review research proposals pitched directly by students and either accept & register or decline them.
Preconditions:	1. The Project Supervisor is logged in. 2. At least one student has submitted a self-proposed project to this supervisor.
Postconditions:	The proposal status is updated, and the student is either registered to the supervisor or notified of rejection.
Priority:	High
Frequency of Use:	Several times during an active application cycle
Flow of Events:	1. The Project Supervisor navigates to the "Proposals from students" tab. 2. The System displays all proposals submitted by students directed to the supervisor. 3. The Project Supervisor reviews the proposal title, writeup, research tags, and supporting documents. 4. The Project Supervisor clicks "Accept & register". 5. The System registers the student under the supervisor and triggers a confirmation notification.
Alternative Flows:	AF-S4: Supervisor declines proposal: 1. The Project Supervisor clicks "Decline". The System marks the proposal as declined and updates the student's dashboard.
Exceptions:	E1: Application cycle deadline passed: 1. The System blocks registration actions and displays a deadline lapsed notice.

Includes:	-
Special Requirements:	Accepting a proposal must automatically trigger an email/system notification to the student.
Assumptions:	A student can only be registered to one confirmed URECA project per cycle.
Notes and Issues:	Direct supervisor acceptance bypasses the automated matching algorithm round.

IV. For Functional Requirement #4 (User)

IV.I ViewNotifications

Use Case ID:	#4-1		
Use Case Name:	ViewNotifications		
Created By:	Edwin Tan	**Last Updated By:**	Edwin Tan
Date Created:	29th August 2026	**Date Last Updated:**	29th August 2026

Actor:	User (Student, Project Supervisor, Administrator)
Description:	Allows an authenticated user to view incoming and historical system notifications regarding status changes, application updates, cycle dates, and matching outcomes.
Preconditions:	The User is logged in and authenticated.
Postconditions:	Notifications are displayed, and selected unread items are updated to a read state in persistent storage. .
Priority:	Medium
Frequency of Use:	High (frequently checked during active application and matching periods)

Flow of Events:	1. The User clicks on the "Notifications" tab in the navigation menu. 2. The System queries and displays the list of relevant timestamped notifications (e.g., application status changes, new applicant alerts, match results, or deadline updates). 3. The User selects a notification to view its complete message details. 4. The System marks the selected notification as read and updates the unread badge counter.
Alternative Flows:	AF-S3: User navigates via notification interactive link The User clicks an embedded link inside a notification (e.g., "View Application" or "View Match Result"). The System redirects the User directly to the target record page
Exceptions:	E1: Notification service fetch timeout or server failure 1. The System displays an error message informing the user that notifications could not be retrieved. 2. The System provides a "Retry" action.
Includes:	-
Special Requirements:	In-app notifications must be delivered and displayed within 1 minute of the triggering event
Assumptions:	All relevant status-change triggers automatically persist notification records for affected user accounts.
Notes and Issues:	Email notifications, where applicable, are dispatched asynchronously within 15 minutes of the event.

IV.II SignIn

Use Case ID:	#4-2

Use Case Name:	SignIn		
Created By:	Ian Tan	**Last Updated By:**	Ian Tan
Date Created:	31th August 2026	**Date Last Updated:**	31th August 2026

Actor:	User (Student, Project Supervisor, Administrator)
Description:	Allows a registered, verified user to securely access the URECAMatch platform using their email address and password.
Preconditions:	The User has a registered and activated account.
Postconditions:	The user is authenticated and granted access to their respective dashboard.
Priority:	High
Frequency of Use:	High
Flow of Events:	1. The User navigates to the sign-in page. 2. The System displays input fields for NTU email address and password. 3. The User enters their credentials and clicks "Sign in". 4. The System validates the credentials against the user database. 5. If valid, the System authenticates the user and redirects them to their respective dashboard.
Alternative Flows:	AF-1: User clicks "Forgot password?" - The System redirects the user to the password reset flow.
Exceptions:	E1: Invalid credentials provided. 1. The System displays an error message and prevents access. E2: Account is deactivated.

	1. The System prevents sign-in and displays an error message.
Includes:	-
Special Requirements:	Password input must be masked, with an option to reveal it. Session shall expire after 30 minutes of inactivity.
Assumptions:	Users have already successfully completed the registration and email verification process.
Notes and Issues:	-

IV.III RegisterAccount

Use Case ID:	#4-3		
Use Case Name:	RegisterAccount		
Created By:	Ian Tan	**Last Updated By:**	Ian Tan
Date Created:	31th August 2026	**Date Last Updated:**	31th August 2026

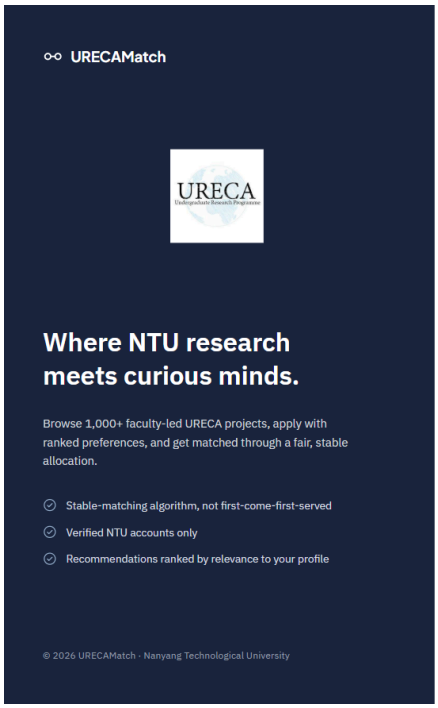
Actor:	User (Student, Project Supervisor, Administrator)
Description:	Allows a user to create a new account using an NTU email address and password, followed by an email verification process to activate the account.
Preconditions:	None (User is not yet registered).
Postconditions:	A new user account is created in the system and activated upon successful verification.
Priority:	High
Frequency of Use:	High (at start of application cycle)

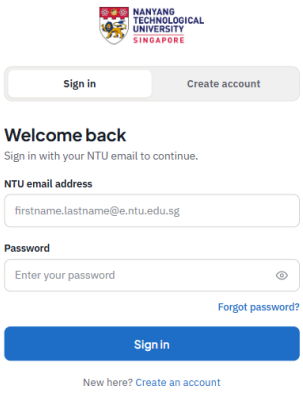
Flow of Events:	1. The User navigates to the registration page. 2. The System displays input fields for full name, NTU email address, password, and account role selection. 3. The User enters details and clicks "Create account". 4. The System validates the email domain and checks for existing accounts. 5. The System sends a verification code to the provided email address. 6. The User enters the verification code. 7. The System verifies the code and activates the account.
Alternative Flows:	AF-1: User requests a new verification code if the previous one expired or was not received.
Exceptions:	E1: Invalid NTU email domain provided. E2: Email address already associated with an existing account. E3: Incorrect or expired verification code entered.
Includes:	-
Special Requirements:	Registration is restricted to NTU email domains (@e.ntu.edu.sg or @ntu.edu.sg) or approved external domains. Password input must be masked.
Assumptions:	Users have access to their provided email account to receive the verification code.
Notes and Issues:	-

4. UI Mockups

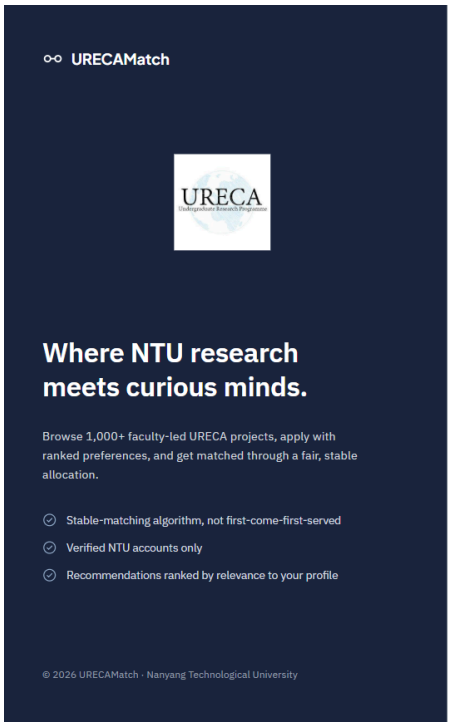
4.1 Shared Screens (All Users)

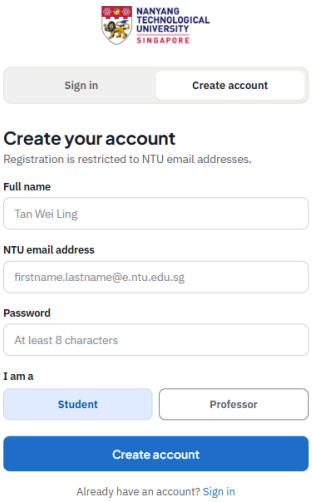
4.1.1 Sign In



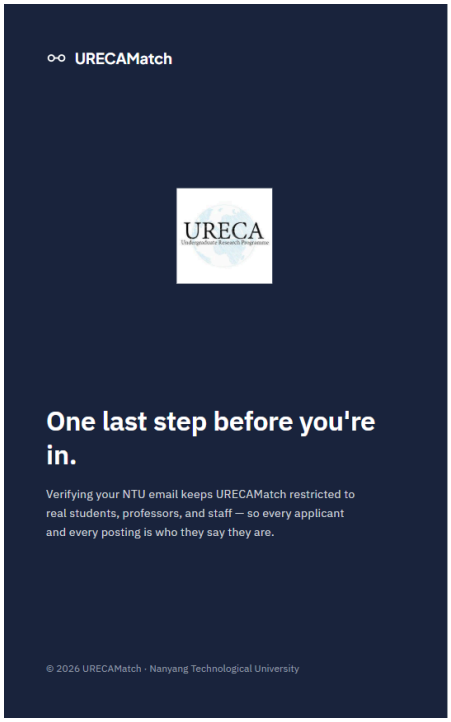


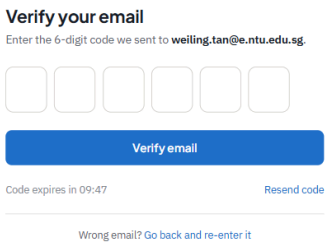
4.1.2 Create Account





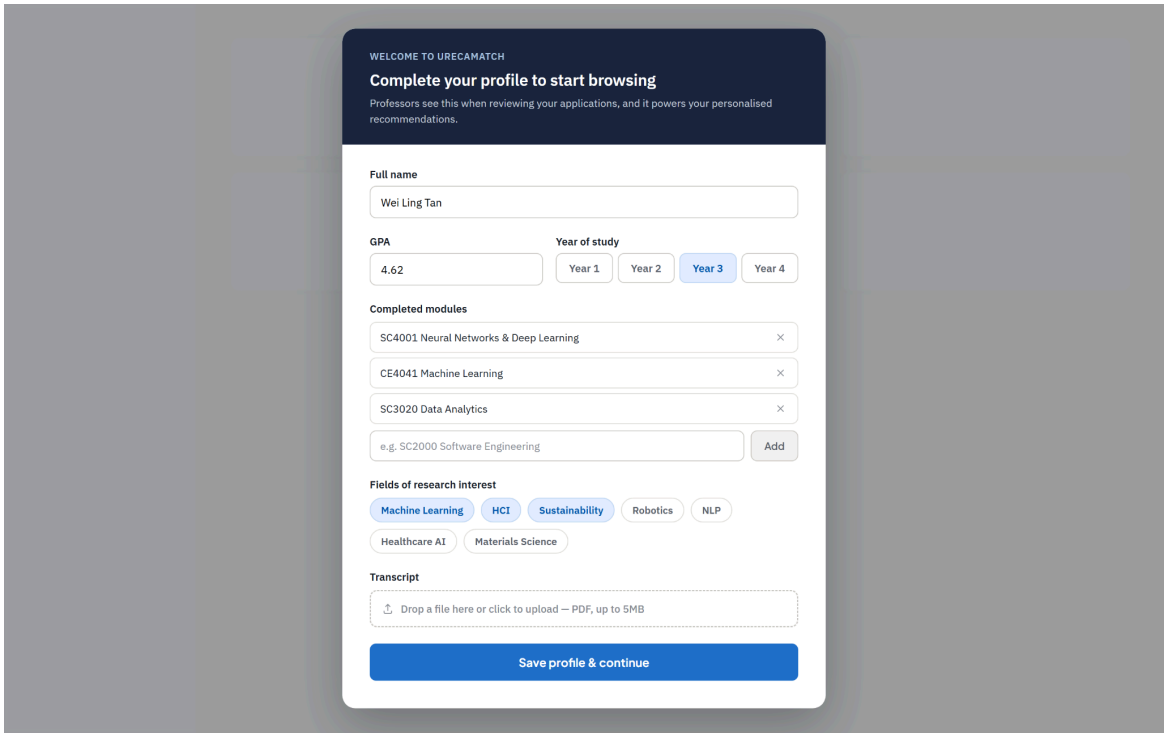
4.1.3 Email Verification



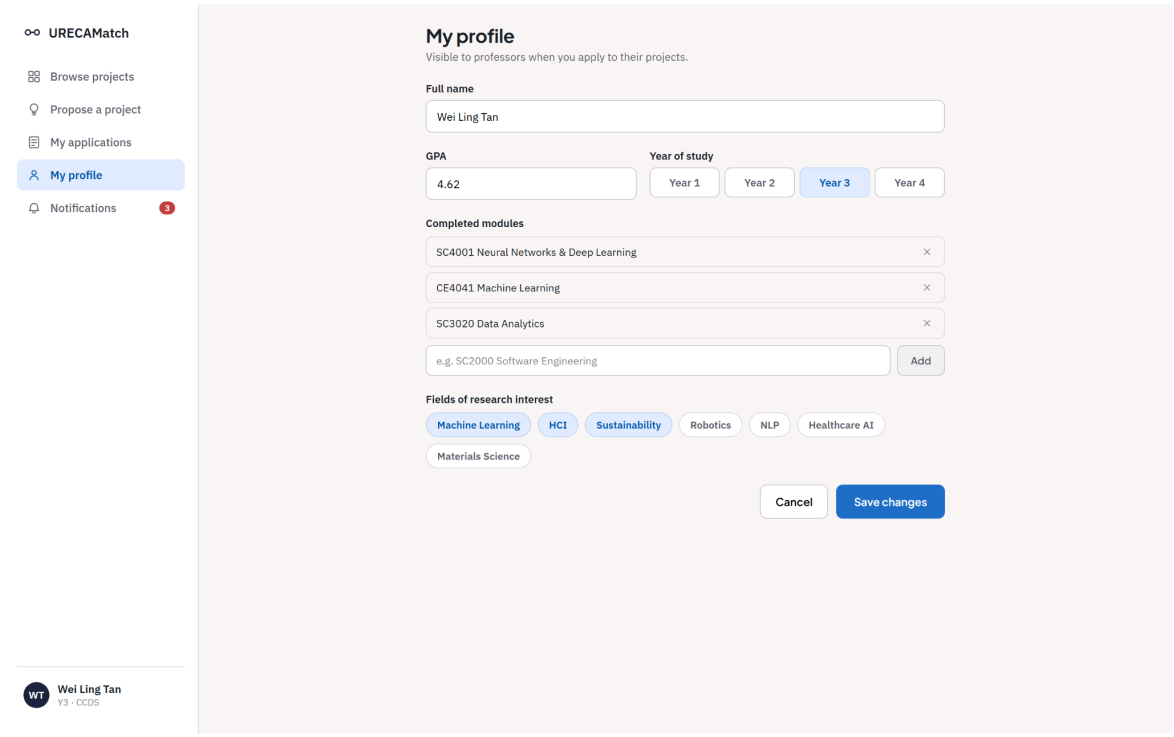


4.2 Student Interface

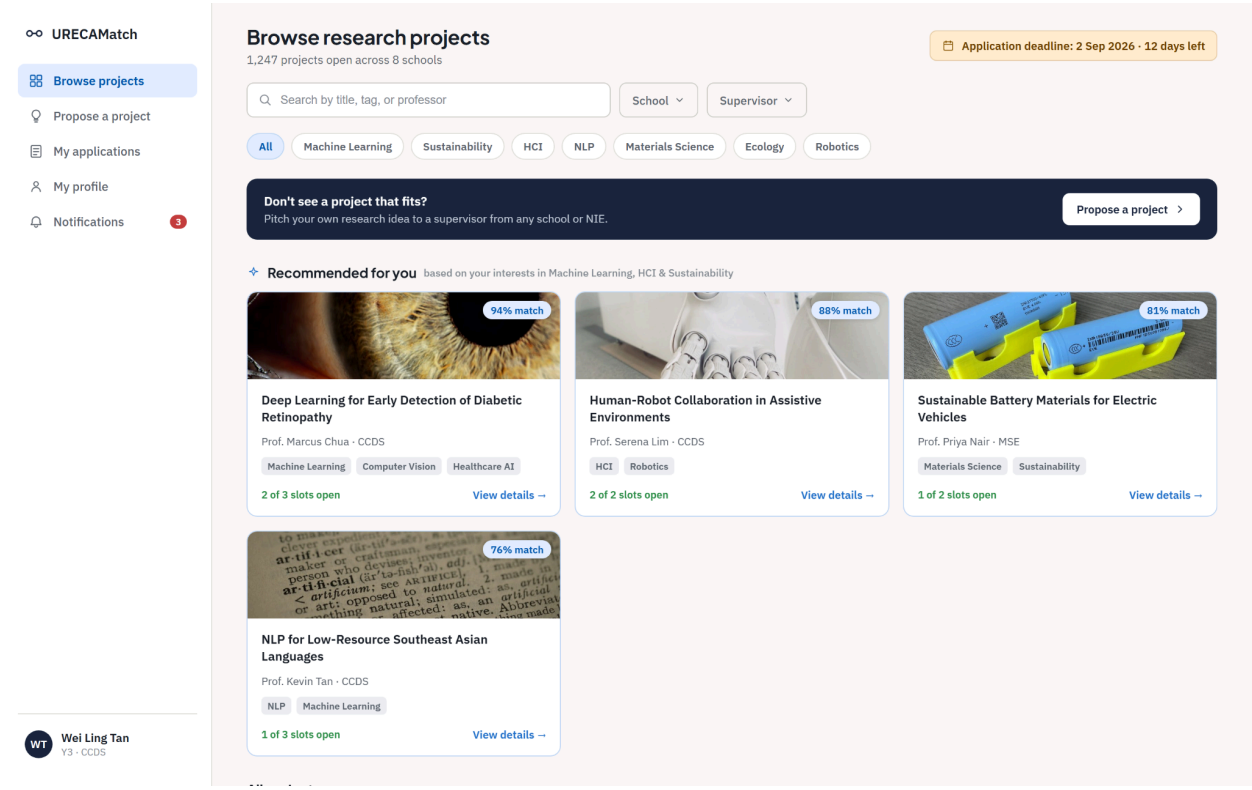
4.2.1 Complete User Profile



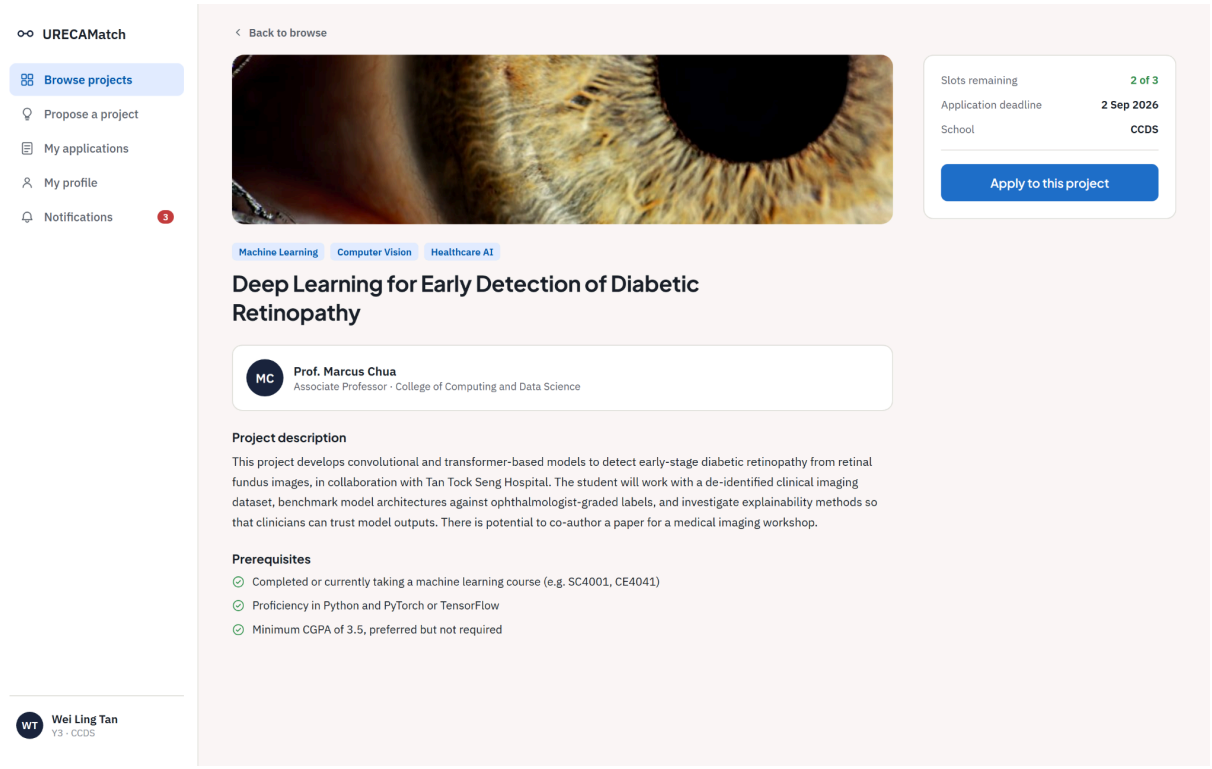
4.2.2 Edit Profile



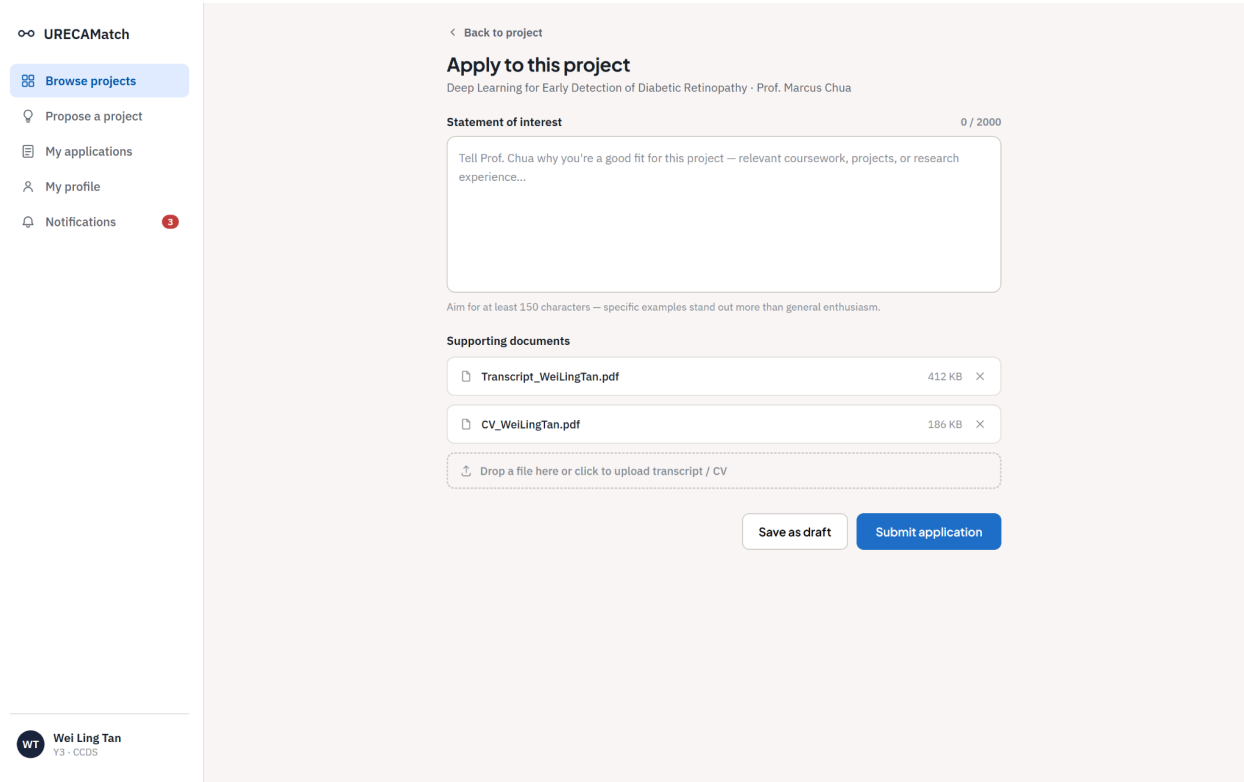
4.2.3 Browse Projects



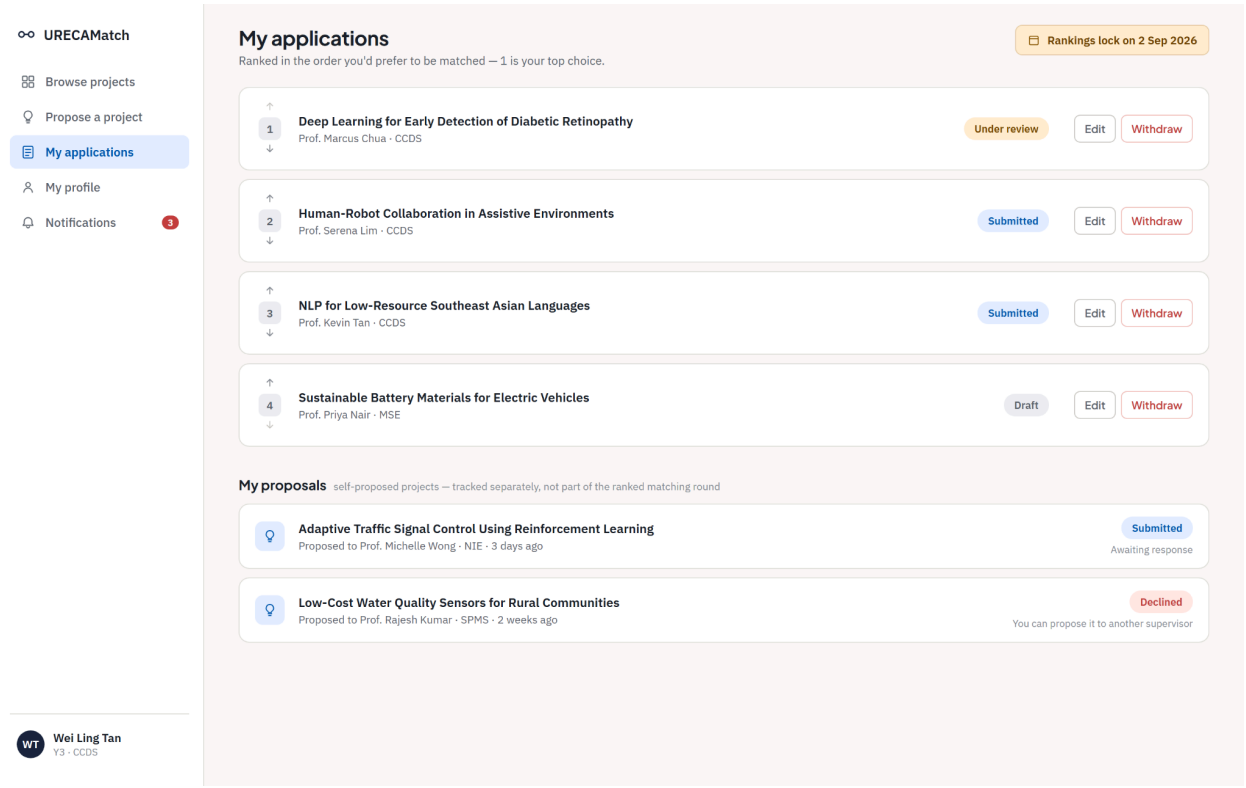
4.2.4 Project Details



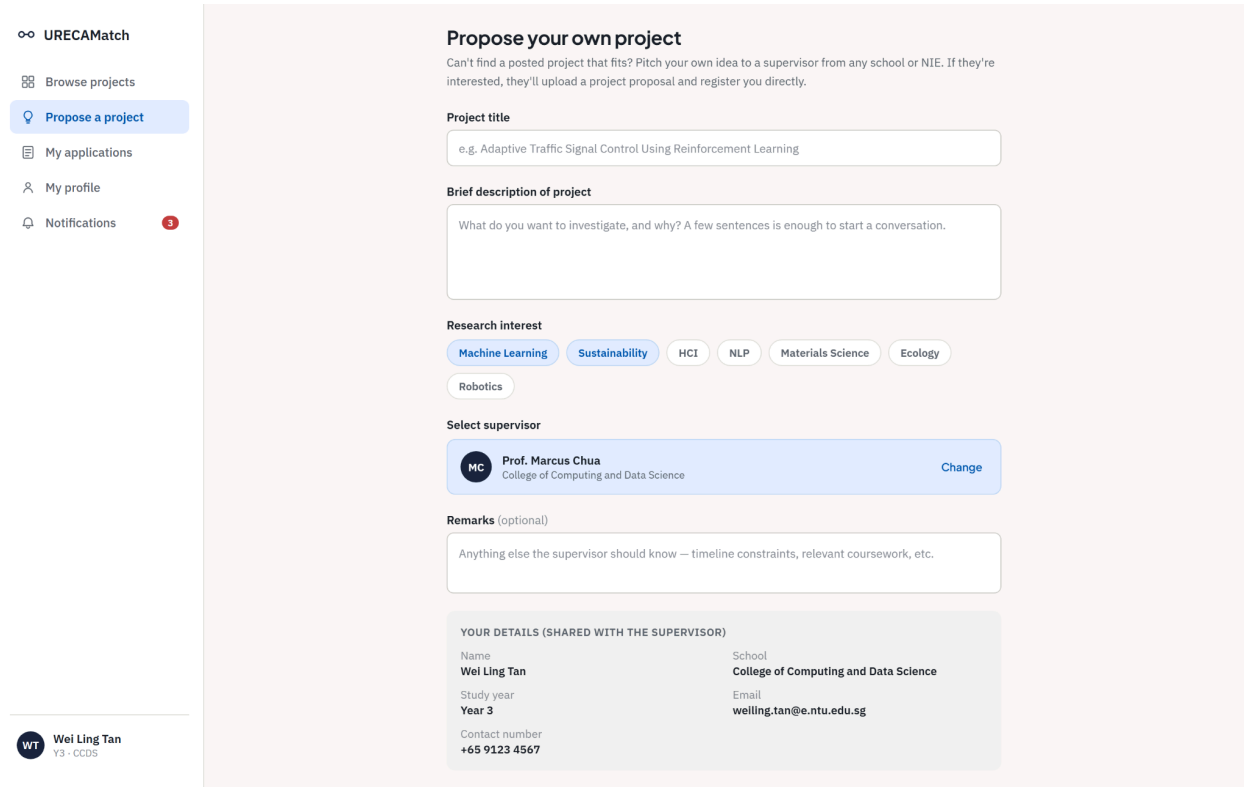
4.2.5 Apply Project



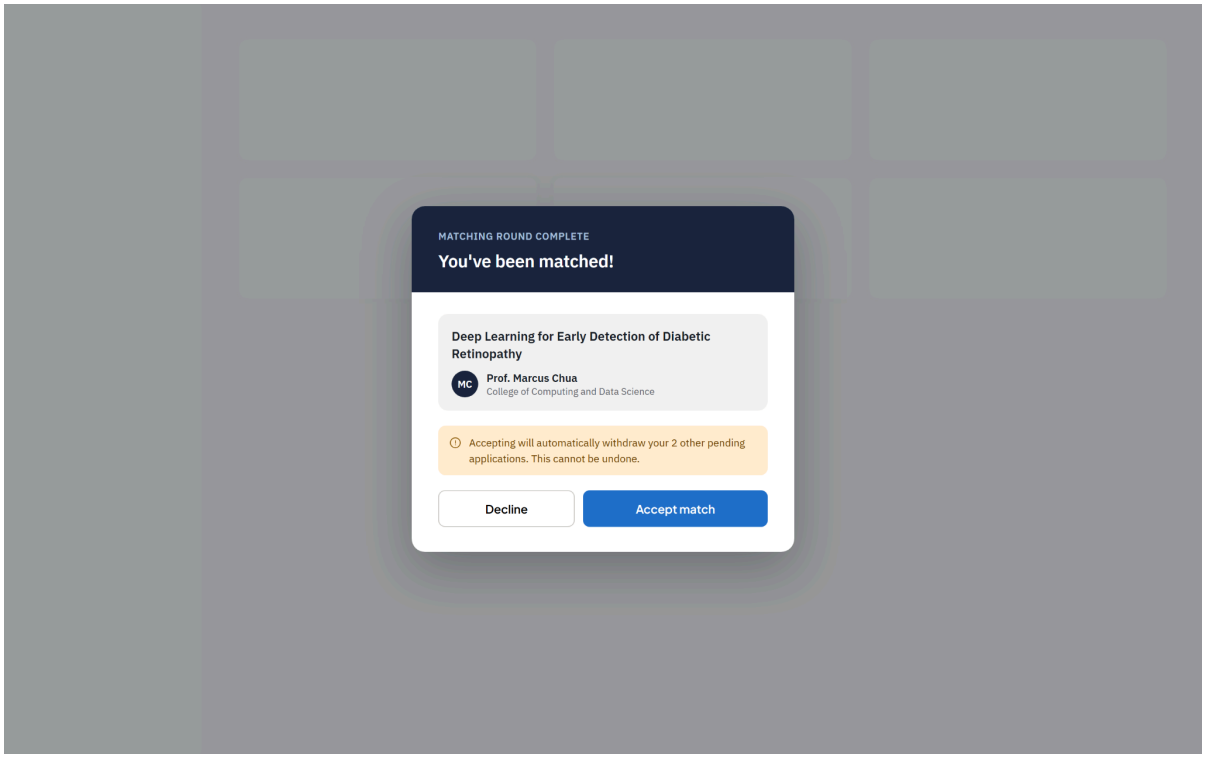
4.2.6 View Application / Rank Application



4.2.7 Self Propose Project

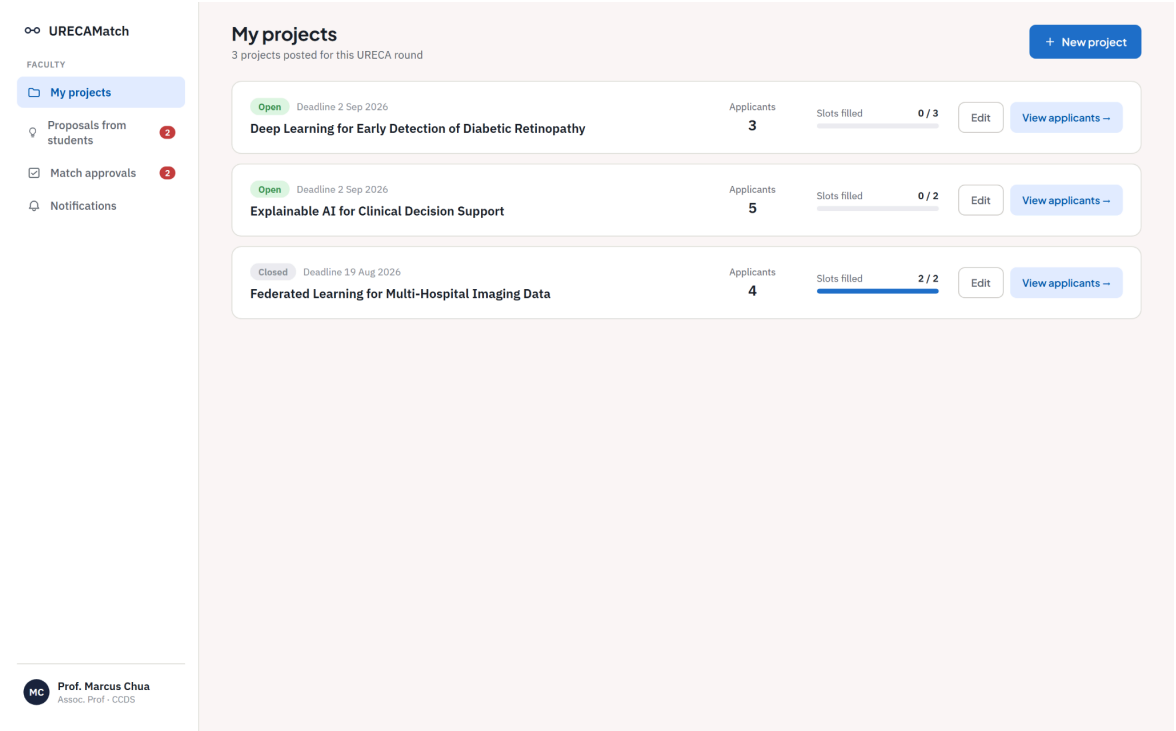


4.2.8 Application Result

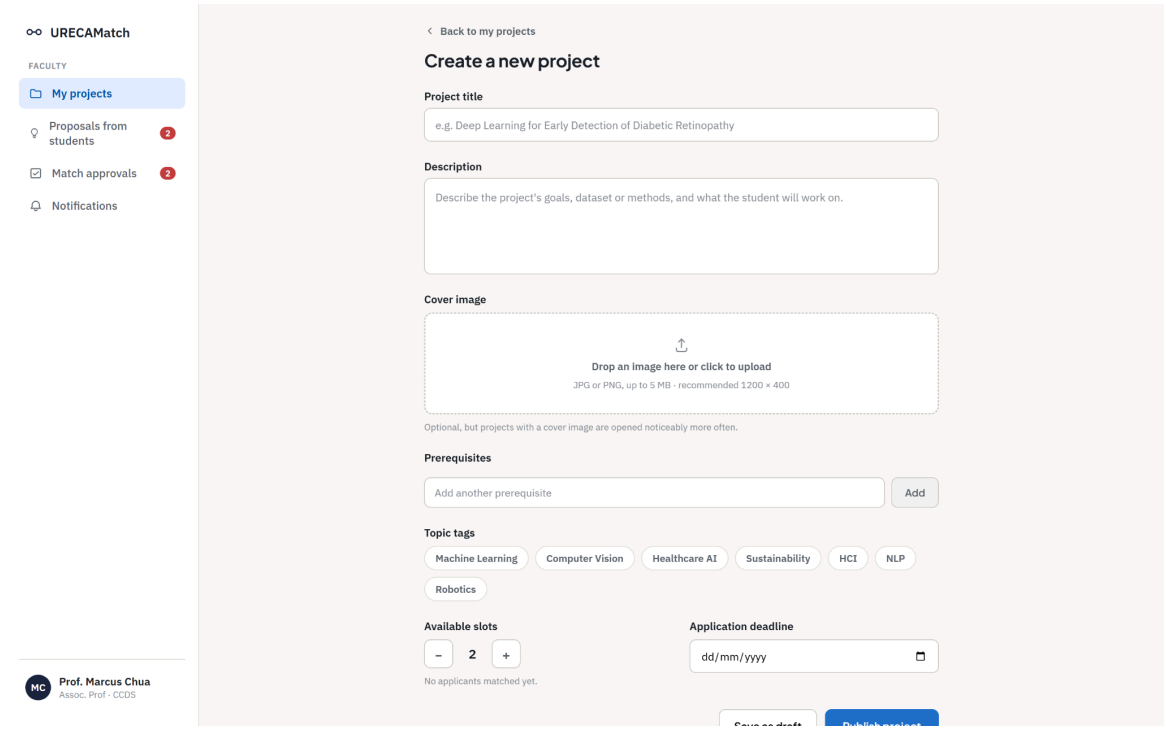


4.3 Project Supervisor

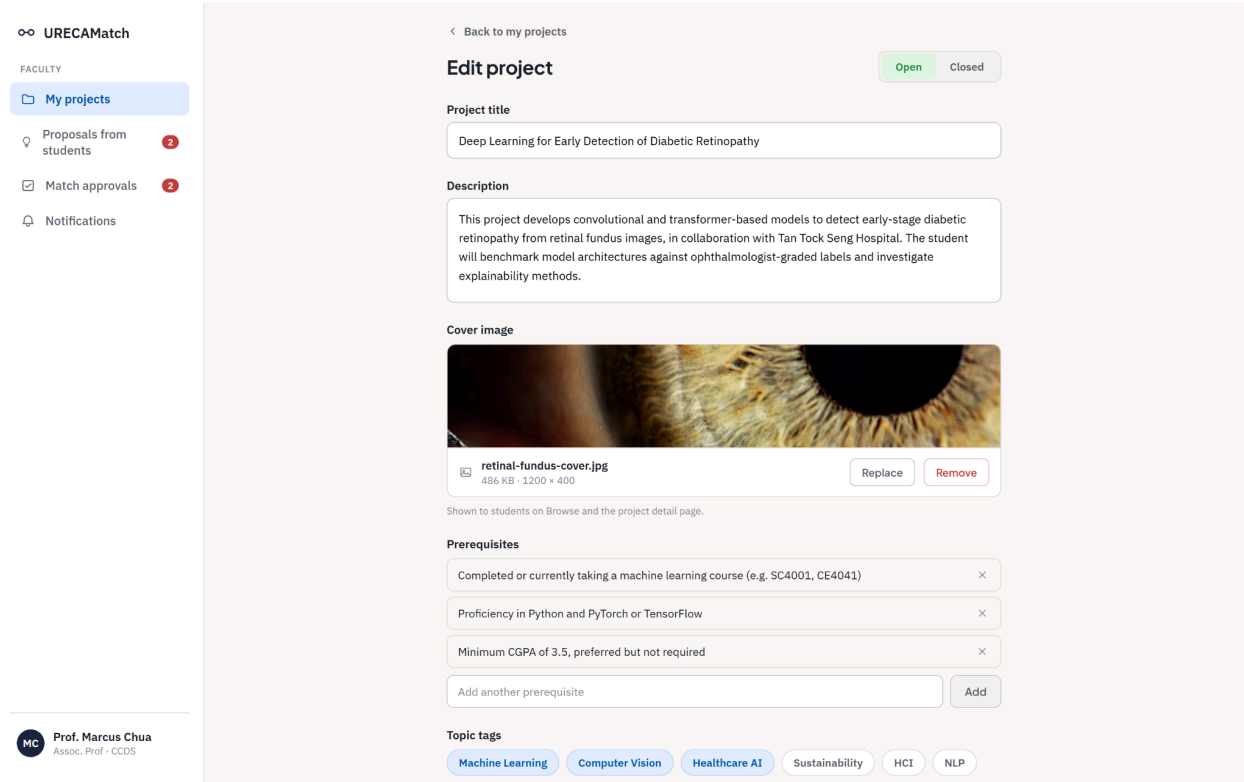
4.3.1 View Projects



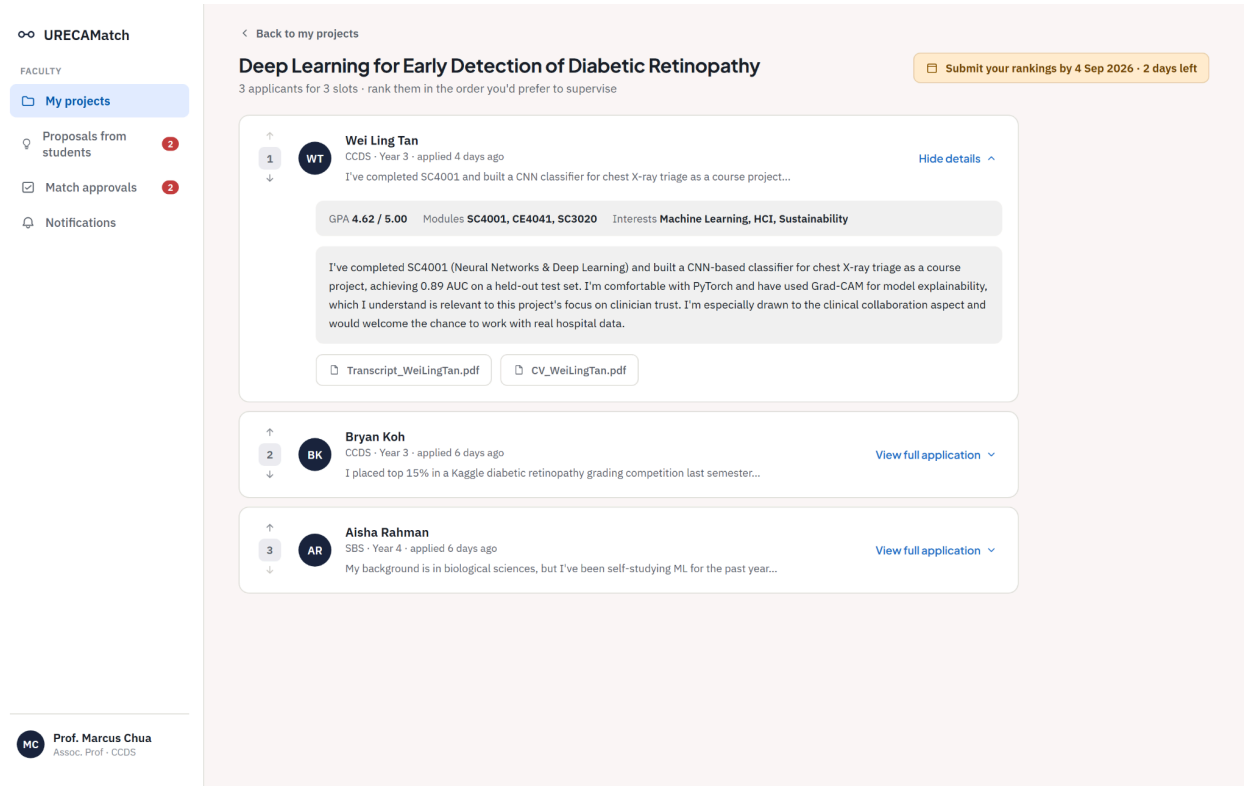
4.3.2 Create Project



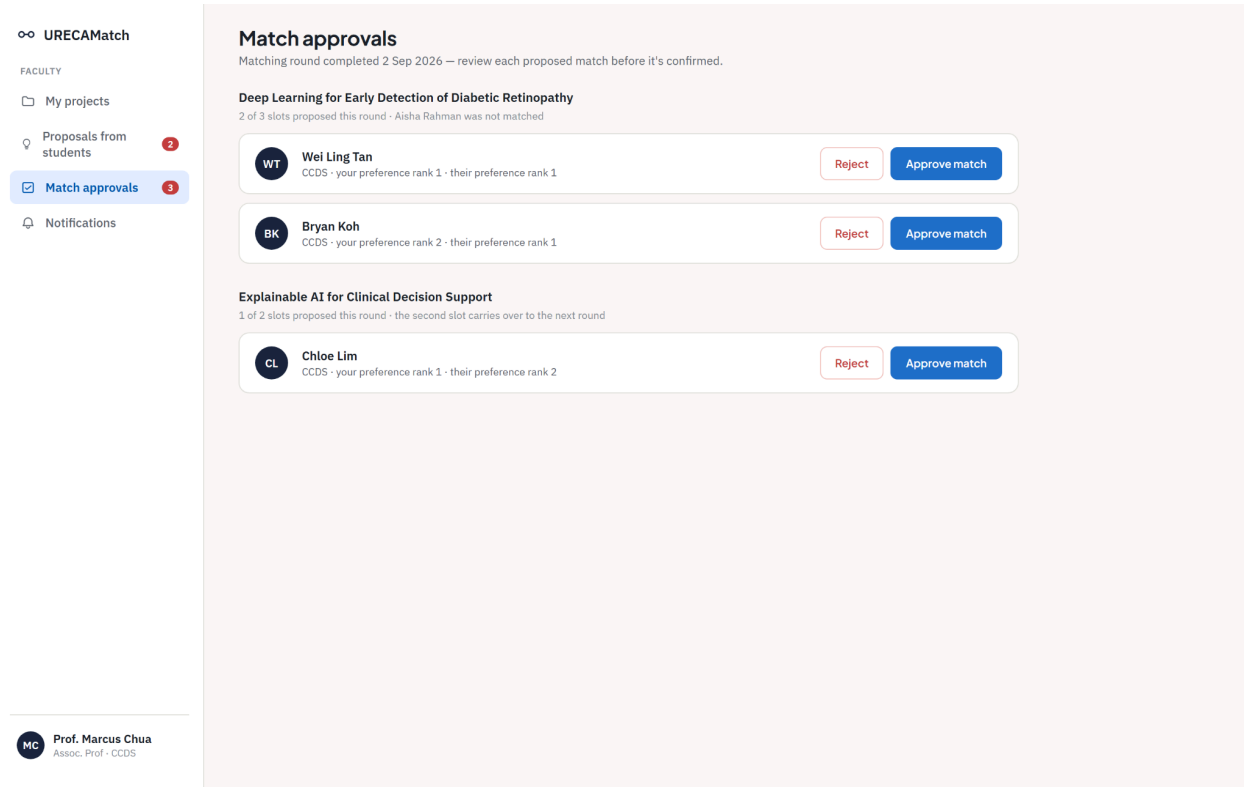
4.3.3 Edit Project



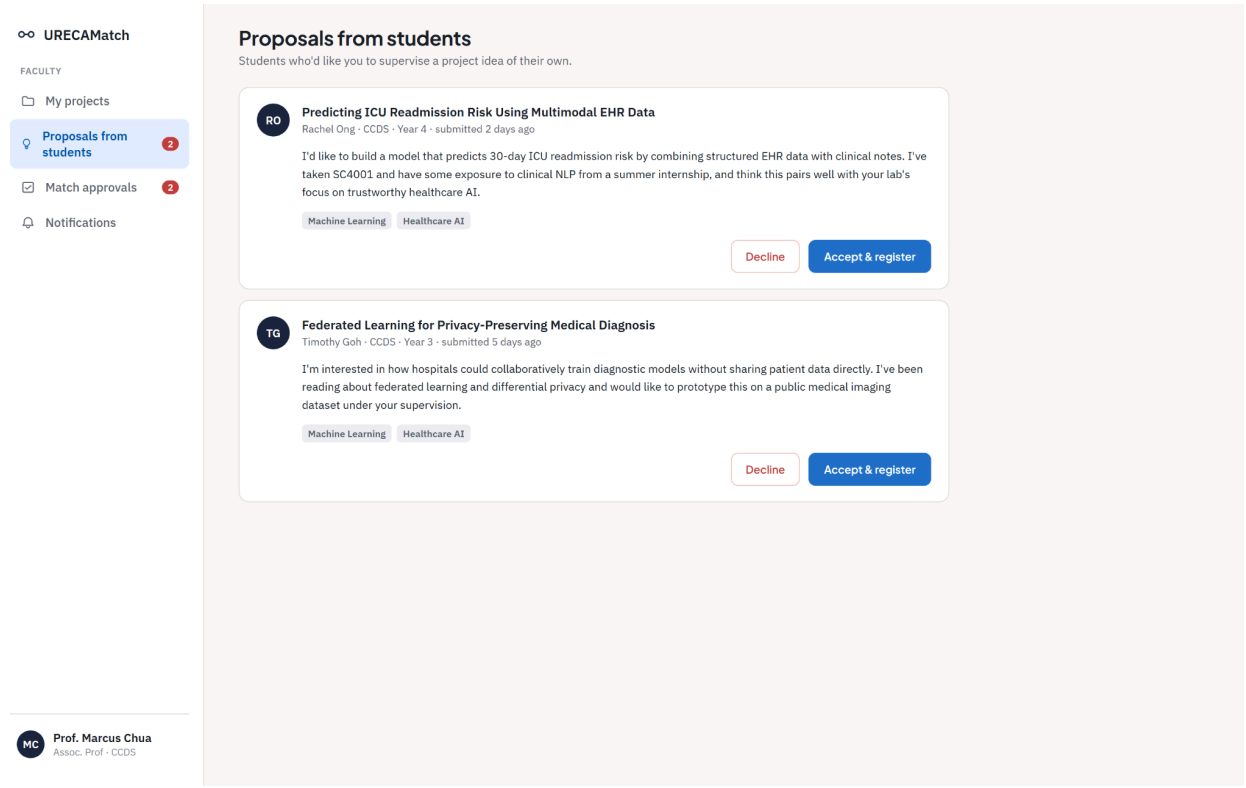
4.3.4 Rank Applicant and view applicant details



4.3.5 Project Approval

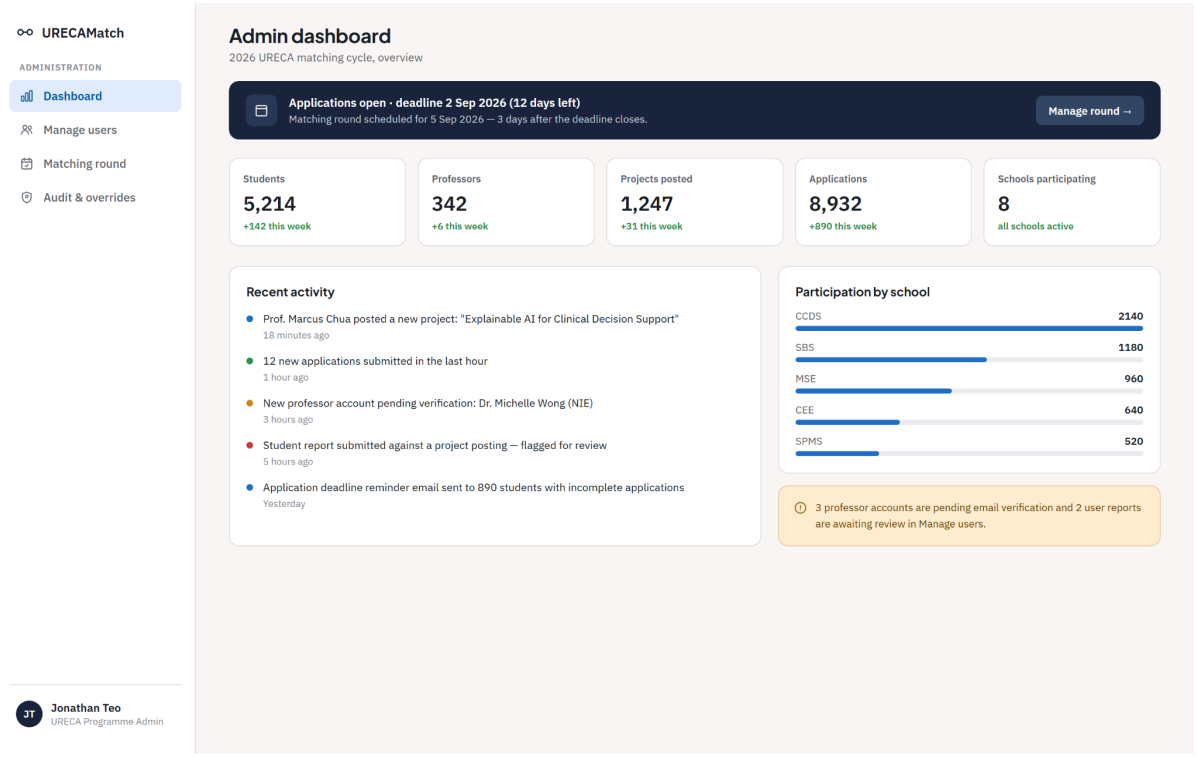


4.3.6 Student Self Propose Project Approval

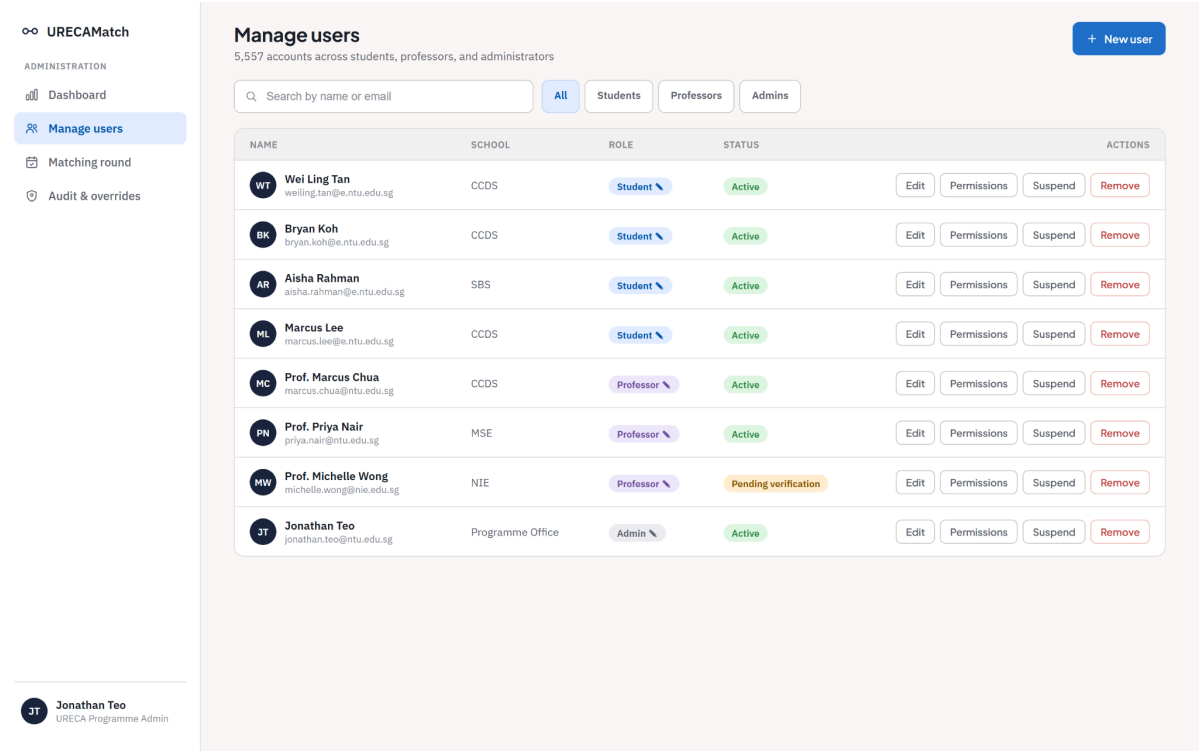


4.4 Admin

4.4.1 Admin Dashboard



4.4.2 Manage Users



4.4.3 Trigger Matching Result

